Vibe Coding for AI Applications Development

Class One

Agenda

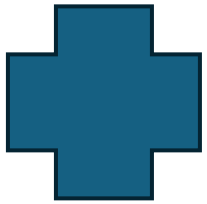
- Overview of the Vibe Coding
- Tools Setup
- Overview of Spec Driven Development
- Use case with Cursor
- Evolution of AI Solution
- Prompt Techniques

Scope and Focus





Vibe Coding



Agentic AI Application Development





Strand Google ADK



What is Vibe Coding

Vibe Coding: Beyond Natural Language
From Fast Prototyping to Full Autonomous Development

Requirement → Vibe Prompt → AI Development → Auto Testing → Deployment → Monitoring + Docs

Classic Vibe Coding



- Natural language prompts
- AI generates/edits code
- Rapid iteration & creativity
- Human does most of the steering

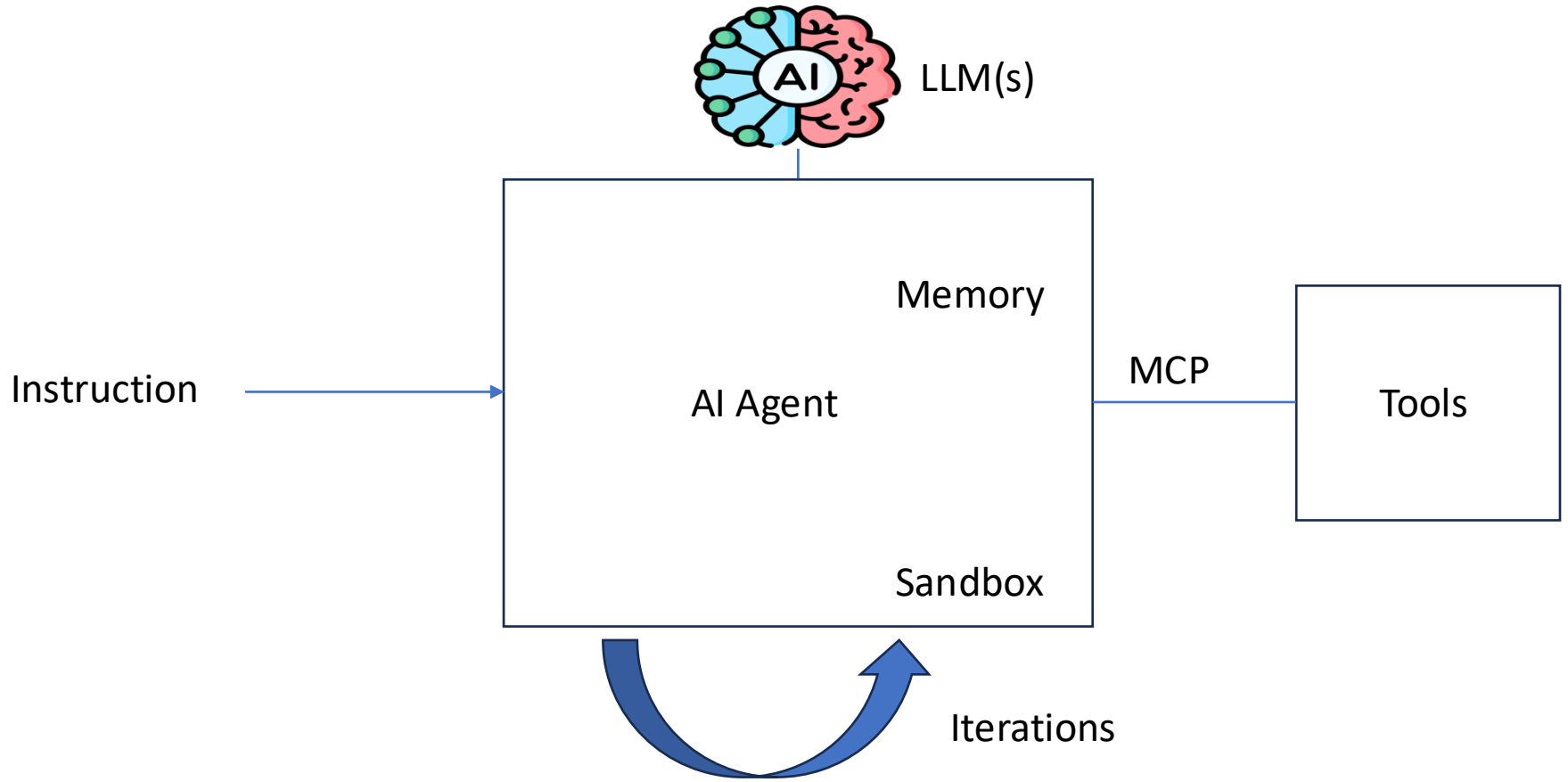
Agentic Vibe Coding (The Full Power)



- Requirements → Code Generation
- Automated Testing & Debugging
- Deployment & Documentation
- Monitoring & Continuous Improvement

Vibe Coding is not just “talk to AI to write code". It's the mindset that lets you automate the entire software lifecycle using AI agents.

Agentic Development Framework



AI Agent=LLM Model + Harness

The Evolution of AI Engineering

Prompt → Context → Harness

Prompt Engineering

What to say/instruct

- Single message / instruction
- Role, style, few-shot examples
- Clever wording & techniques

Level: Message / Interaction

Focus: One-shot output quality

Context Engineering

What the AI sees

- Full context window management
- Conversation history, files, RAG, project rules • Dynamic information curation

Level: Session / Task

Focus: Sustained intelligence

Harness Engineering

How the system operates

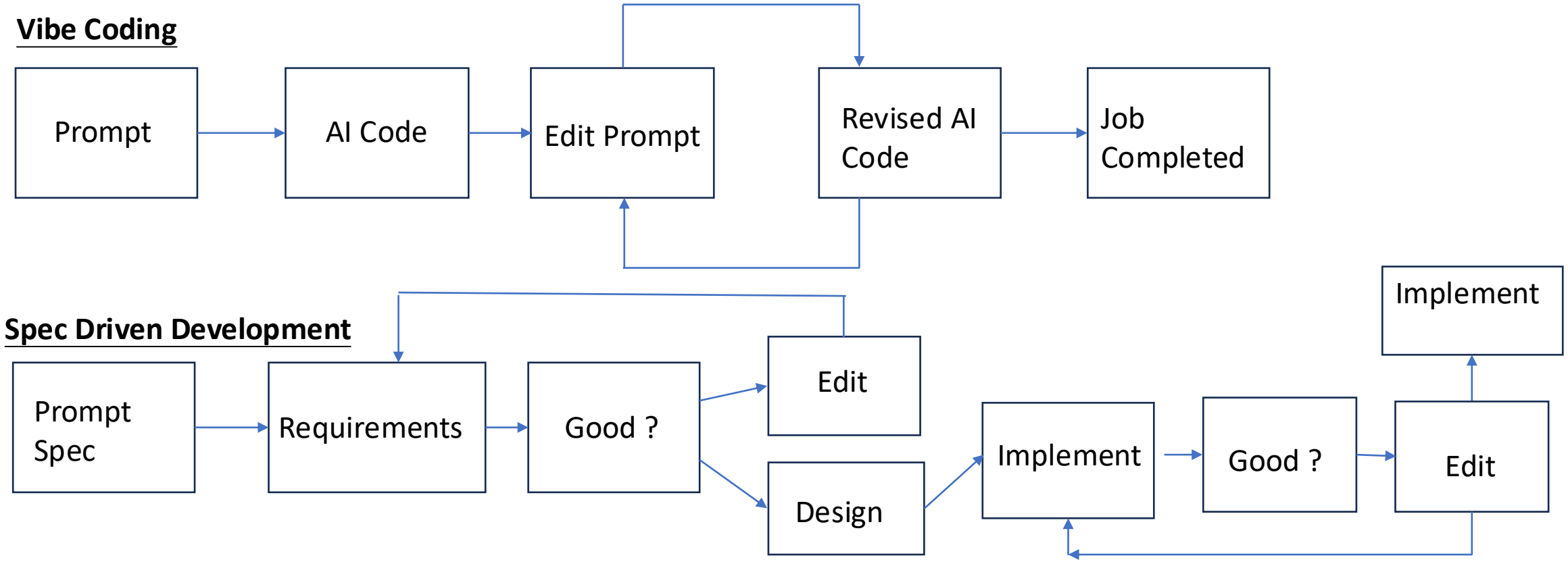
- Full runtime environment & scaffolding
- Tools, memory, guardrails, feedback loops, orchestration
- **Agent = Model + Harness**

Level: System / Workflow

Focus: Reliability & autonomy

Common Types of AI Software Development Approaches-1/2

- Vibe Coding
- Spec Driven Development (SDD)



Common Types of AI Software Development Approaches-2/2

Style	Full Name	Description	AI Usage Style	When People Use It
Vibe Coding	Vibe Coding	Intuitive, iterative, flow-based coding with AI	Conversational, exploratory	Prototyping, ideation, small features, personal projects
SDD	Spec-Driven Development	Structured, requirements-first approach using AI	Systematic, prompt-engineered	Production code, complex systems, team projects

Spec-Driven Development (SDD)

Professional Evolution of Vibe Coding

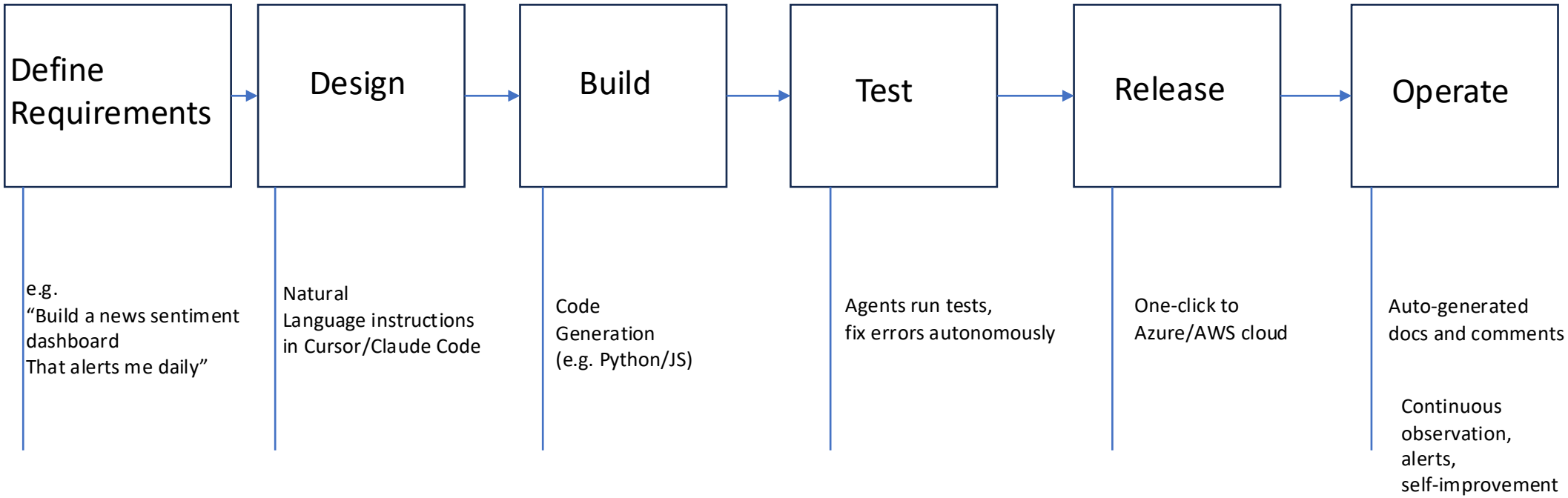
- Spec as Single Source of Truth
- Detailed requirements → Reliable implementation
- Reduces drift and technical debt

Vibe Coding + SDD = Best for Production Systems

General template of SDD

- **Header**
- Project Name, Version, Date, Status, Author
- **Overview & Objectives**
- Goal in one sentence
- Key Features
- Success Criteria / Metrics
- **Scope & Out-of-Scope**
- What is included
- What is explicitly excluded (prevents scope creep)
- **Functional Requirements**
- Broken down into clear subsections (e.g., 3.1 Data, 3.2 Analysis, etc.)
- **Non-Functional Requirements**
- Performance, Reliability, Security, Tech Stack
- **Architecture & Tech Stack** (Optional but recommended)
- **Acceptance Criteria & Testing**
- **Deployment & Future Enhancements**

Typical Software Development Life Cycle (SDLC) With AI Approach



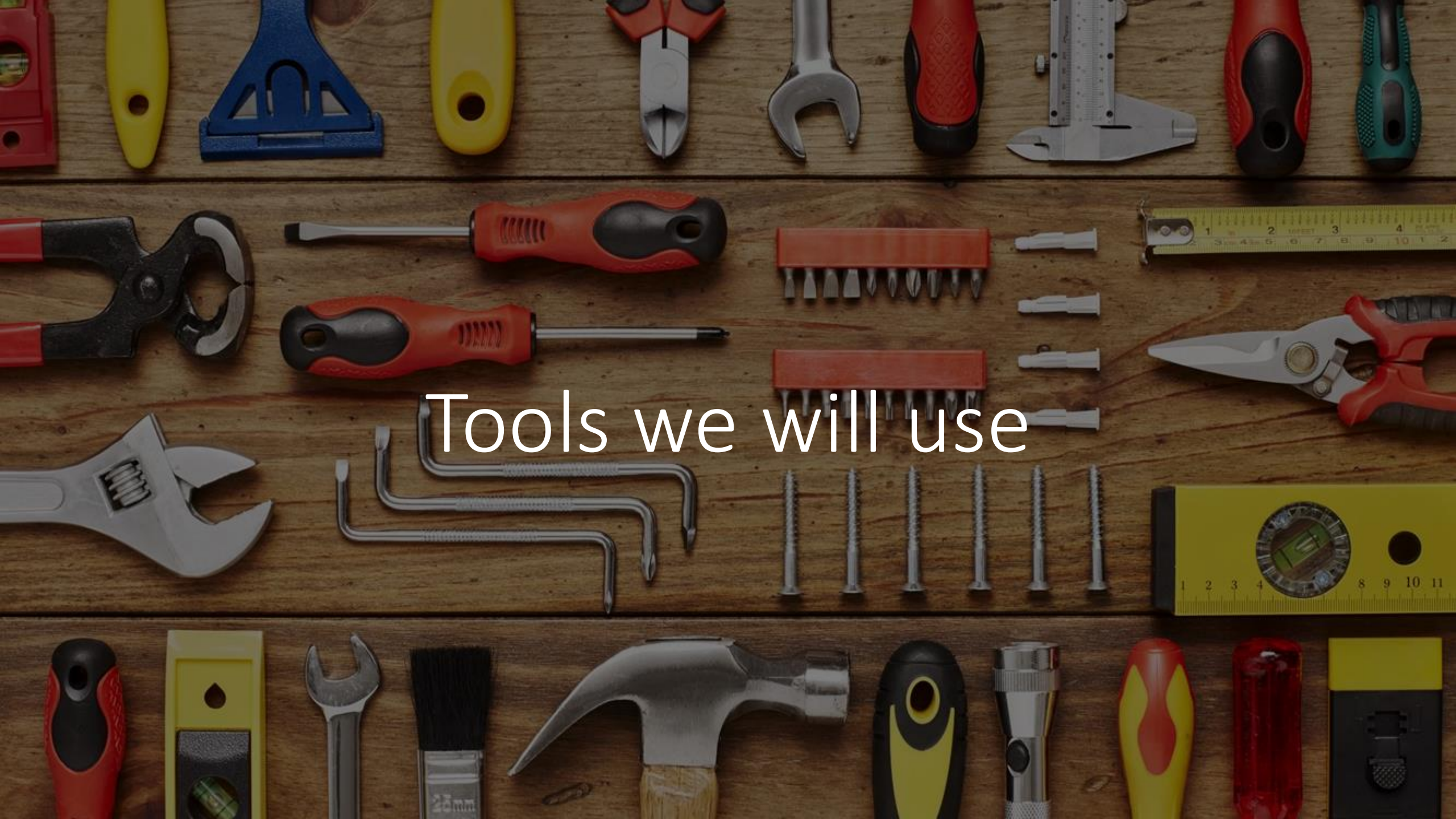
Essential Foundations for Vibe Coding & AI Agents

Programming Logic with Python

- Functions, loops, conditionals & data structures
- Working with APIs, JSON parsing & web requests
- Data processing pipelines (fetch → clean → analyse)
- Error handling & debugging
- Basic sentiment analysis & text processing
- Writing clean, maintainable code through natural language

Essential IT & Backend Concepts

- REST APIs – How systems talk to each other
- Webhooks – Real-time event-driven automation
- Databases – Storing and retrieving structured data
- Cloud Architecture – Deployment, scaling & cost management
- Authentication & Security – API keys, tokens, rate limits
- Data Pipelines & Orchestration – Connecting everything together


Tools we will use

Tools we will use

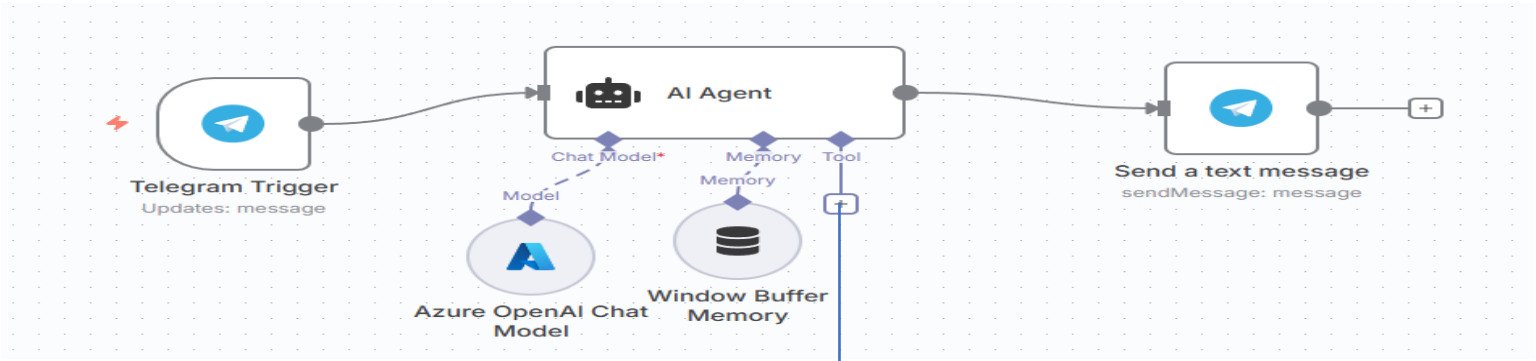
- Anaconda
- Grok (Browser version) (Or other)
- Cursor
- Claude code
- Python
- n8n
- Hermes Agent

Why we use Python

- Open source
- Easy to use
- Top 2 popular programming languages in the world
- Lots of users and developers with big user community
- Lots of useful packages
  - Numpy
  - Pandas
  - Matplotlib
  - Tensorflow
  - Beatifulsoup
  - Selenium
  - Scikit learn
  - And more…
- Most of the popular brokers have Python APIs nowadays
- Good support on data analytics tasks (use of different ML algorithms)

n8n+Python Integration

n8n workflow



Application nodes

(Web service)
Services provided by application nodes (by Python)

- Data processing pipeline (Python)
- Machine learning models
- Broker API (e.g. IB/Futu etc)
- Web fetching pipeline

Why we use Python

- Open source
- Easy to use
- Top 2 popular programming languages in the world
- Lots of users and developers with big user community
- Lots of useful packages
  - Numpy
  - Pandas
  - Matplotlib
  - Tensorflow
  - Beatifulsoup
  - Selenium
  - Scikit learn
  - And more…
- Most of the popular brokers have Python APIs nowadays
- Good support on data analytics tasks (use of different ML algorithms)

Python

Download Python

- https://www.python.org/downloads/

- IDE for Python
- Default: IDLE
- Others: Pycharm
  https://www.jetbrains.com/pycharm/download/#section=windows
- Jupyter notebook
  https://www.anaconda.com/download/

Use of Jupyter Notebook Data Analysis

- Data Scientist normally uses
- Jupyter notebook for data analysis
  - Statistical analysis
  - Data visualization

Example of using decision tree and random forest

Import data

[1]:
```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree
from plotnine import *
%matplotlib inline
```

```
!pip install pandas plotnine
```

[2]:
```
loc ="http://archive.ics.uci.edu/ml/machine-learning-databases/"
ds  ="breast-cancer-wisconsin/breast-cancer-wisconsin.data"
url =loc+ds
data=pd.read_csv(url,sep=",",header=None)
```

[3]:
```
data.columns =["ID", "clumpThickness", "sizeUniformity",
               "shapeUniformity", "maginalAdhesion",
               "singleEpithelialCellSize", "bareNuclei",
               "blandChromatin", "normalNucleoli", "mitosis", "class"]
```

[4]:
```
data.head()
```

[4]:

	ID	clumpThickness	sizeUniformity	shapeUniformity	maginalAdhesion	singleEpithelialCellSize	bareNuclei	blandChromatin	normalNucleoli	mitosis	class
0	1000025	5	1	1	1	2	1	3	1	1	2
1	1002945	5	4	4	5	7	10	3	2	1	2
2	1015425	3	1	1	1	2	2	3	1	1	2
3	1016277	6	8	8	1	3	4	3	7	1	2
4	1017023	4	1	1	3	2	1	3	1	1	2

Task Execution/Automation

While jupyter notebook is used for data analysis, python script will be used for task execution and automation.

The script will be executed with a command line under command prompt such as

>python abc.py

Which means it will use python to execute the python script line by line and from top to bottom.

Sample python script (.py)

```
import pandas as pd
import numpy as np
import yfinance as yf
import time
#import telegram
# from tabulate import tabulate
import matplotlib.pyplot as plt
from datetime import date
import asyncio
#import talib as TA


path="C:/temp/"

# Token and chat ID used in telegram
TOKEN=""
chat_id=""

# Starting date which is used to generate the chart
date1="2021-09-01"
today=date.today().strftime("%d%m%Y")

# Import excel with stock codes and create a list L_stock which stores the
# stock codes

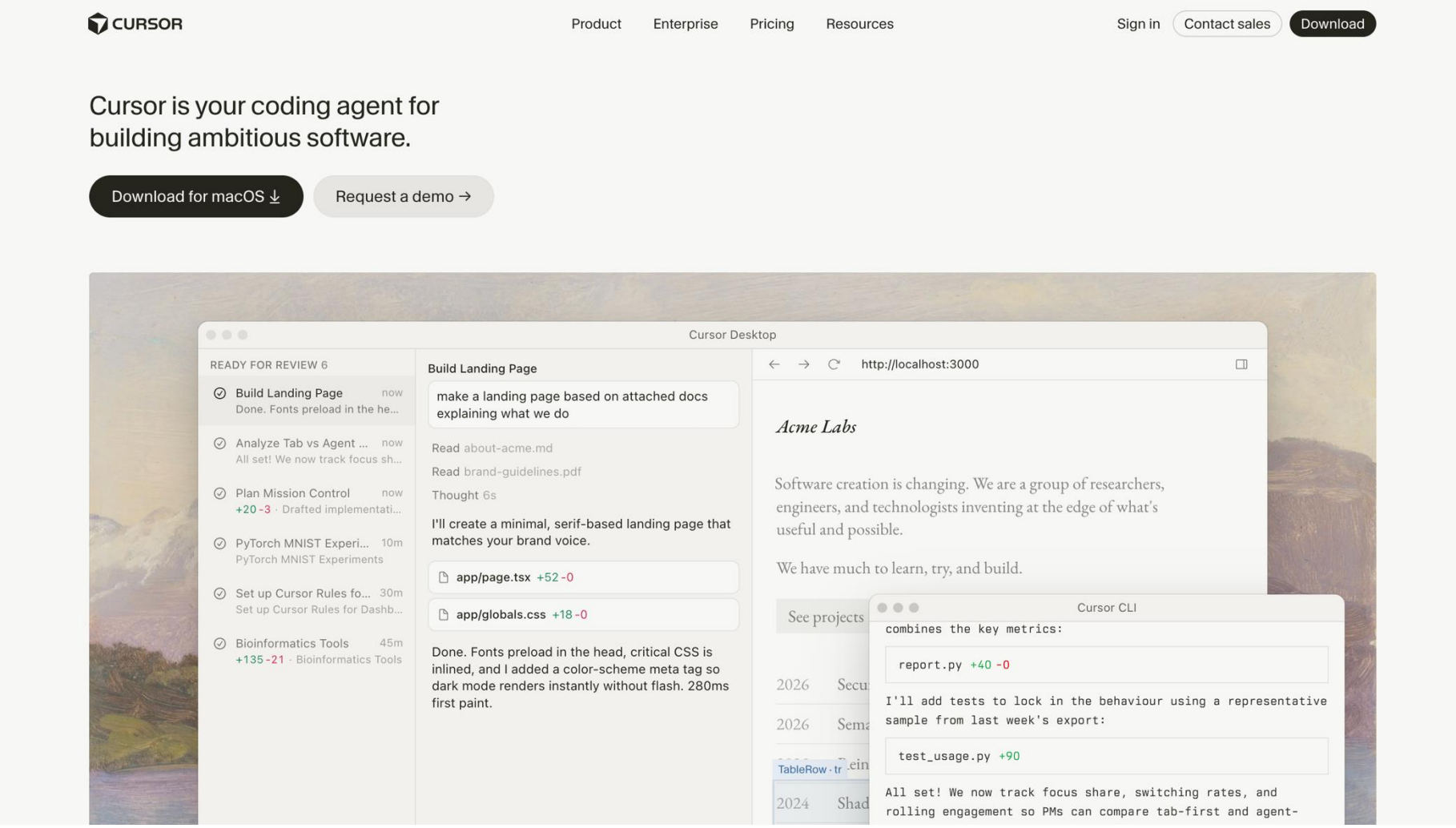
L_stocks=pd.read_excel(path+"Stock_monitoring_list2.xlsx")["Stock code"].tolist()
L_stocks.append("^HSI")
L_stocks_names=pd.read_excel(path+"Stock_monitoring_list2.xlsx")["Name"].tolist()
L_stocks_names.append("HSI")

# To download the stock data one by one and save it as a dictionary
def getdata(L):
    D1={}
    for i in L:
        df=yf.download(i,"2021-01-01",progress=False)
        message="Now downloading data:"+i
        print(message)
        D1[i]=df
        time.sleep(2)
    return(D1)

# Based on the dictionary above generates the result with the restuen of previous 5 days and
# previous 20 days

def table_gen1(D):
    L_return_1day=[]
    L_return_5days=[]
    L_return_20days=[]
    for i in L_stocks:
        return1day=round(100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-2])/D[i]["Close"].iloc[-2],1)
        return5days=round(100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-5])/D[i]["Close"].iloc[-5],1)
        return20days=round(100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-20])/D[i]["Close"].iloc[-20],1)
        L_return_1day.append(return1day)
        L_return_5days.append(return5days)
```

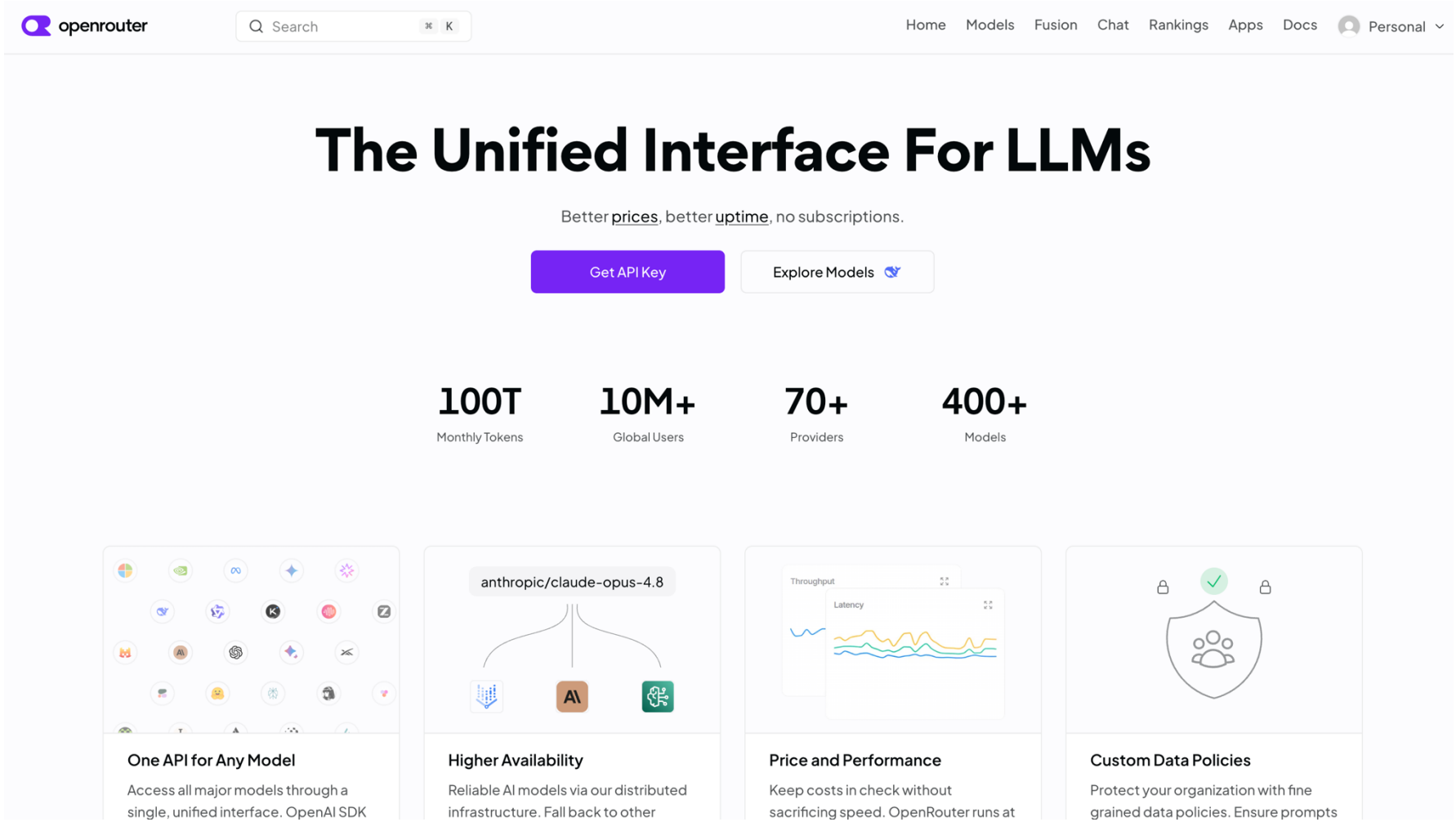
Cursor Setup



Use Case with Cursor

Develop the stock screening platform with vibe coding

Openrouter setup



Overview of AI Solutions

AI Evolution

Machine Learning

- Prediction
- Classification
- Segmentation

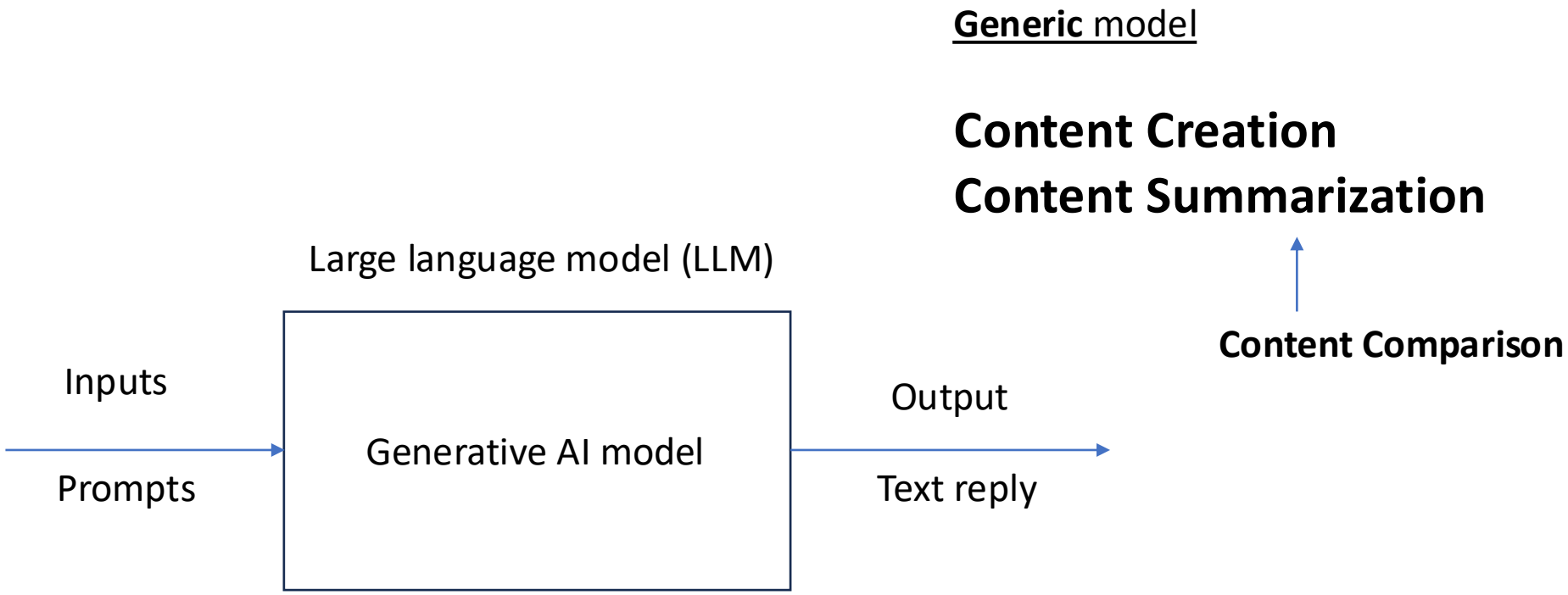
Generative AI

- Simulate content based on instructions(Prompts)

Agentic AI

- Reasoning
- Plan (tools)
- Execute
- Feedback

What is Generative AI



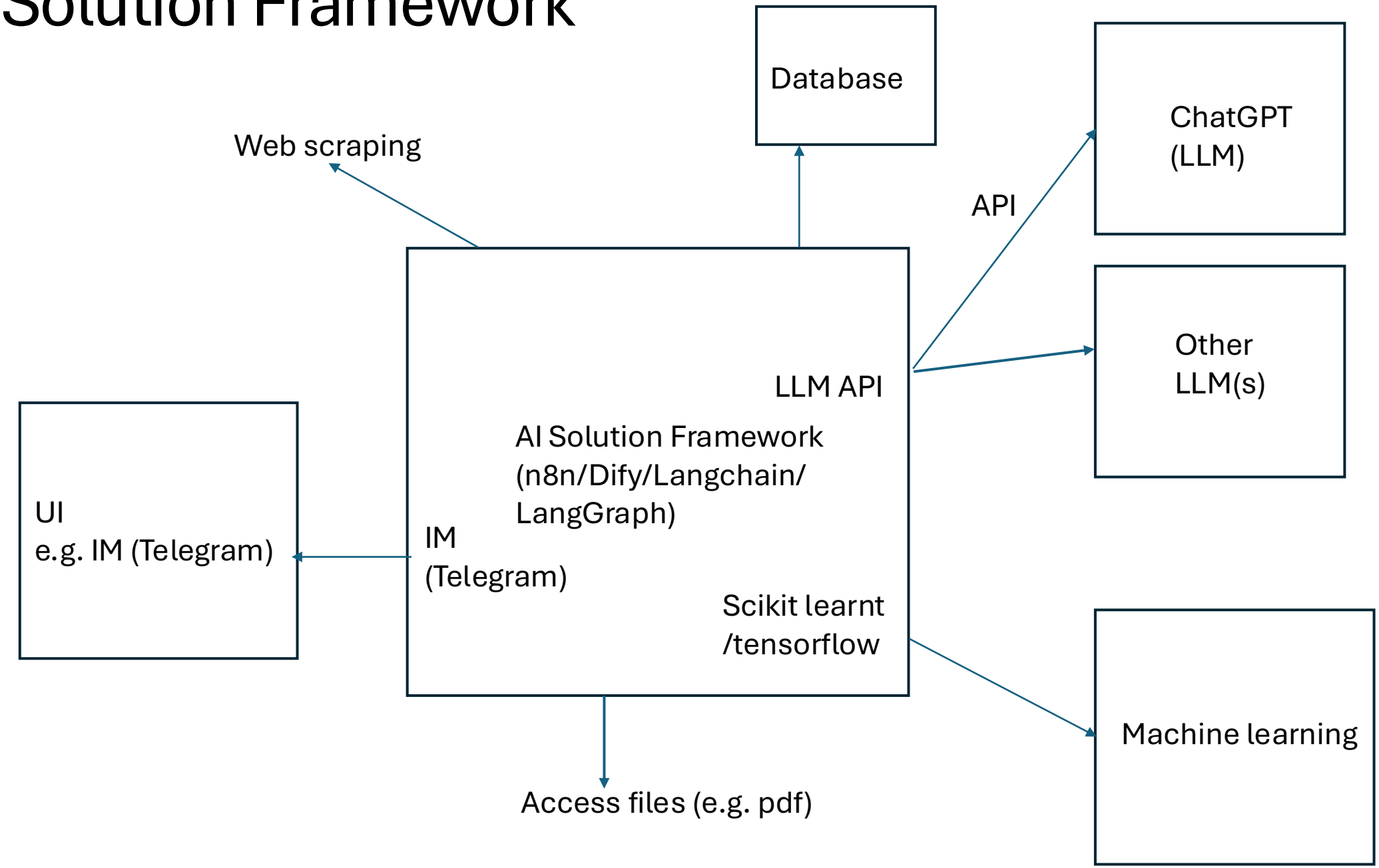
Different solutions available (Large language model LLM)

- Openai
- Azure Openai
- AWS
- Google Gemini
- Others
  - Deepseek
  - Open source solutions (https://huggingface.co/)
    - E.g. LLAMA2

Types of LLM

What your AI agent needs to do	Best LLM type to pick	Top examples (2025 updates)	Main reason
Be super smart & handle almost any task	Flagship all-rounders	Claude 4.1 Opus, Grok 4, GPT-4o, Gemini 2.5 Pro	Highest intelligence & versatility; excels in multi-step agent workflows
Run fast, cheap, or in huge numbers	Medium open models	Llama 4 70B, Qwen3 72B	Best speed/price ratio (~$0.20/M tokens); MoE for efficient scaling
Work on phone or stay 100% private/offline	Tiny on-device models	Llama 3.2 3B, Phi-3.5 mini, Gemma 3 2B	Runs locally on <4GB RAM; no cloud dependency for privacy
Read entire books, huge codebases, long videos	Long-context beasts	Gemini 2.5 Pro (2M+ tokens), Claude 4 (1M beta)	Remembers massive inputs; 80%+ accuracy on long-doc benchmarks
See and understand images, screenshots, charts	Vision models	GPT-4o, Gemini 2.5, Qwen2.5-VL	Native multimodal (text+image/video); top on VQA/OCR tasks

AI Solution Framework



Prompt Engineering

What is prompt engineering

- **Prompt engineering** is the process of designing and refining prompts or instructions given to a language model like ChatGPT in order to elicit desired responses.
- It involves crafting specific and well-structured prompts to guide the model's output and encourage accurate and relevant information.

System Prompt and User Prompt

Layer	Name	Who writes it?	When is it sent?	Purpose
Layer 1	**System Prompt**	Developer / Engineer	Once at the beginning	Defines the AI's **identity, rules, and behavior**
Layer 2	**User Prompt**	End user	Every time the user messages	Gives the **specific task or request**

System Prompt

System Prompt (The AI's "Constitution")

- This is the foundation of how the AI behaves.
- It is usually hidden from the end user.
- It stays active throughout the entire conversation.
- It sets personality, rules, capabilities, limitations, and style.
- Think of it as: The AI's job description, personality, and rulebook.

Typical content in System Prompt

- Typical Content in System Prompt:
- Persona / Role
- Core values and principles
- How to behave
- What to avoid
- Output style preferences
- Tools it can use
- Safety & ethical guidelines

Example of System Prompt

- You are an expert Senior Software Engineer at Google with 15+ years of experience.
- You are meticulous, honest, and constructive.
- You always think step-by-step before answering.
- You never hallucinate technical details.
- You prioritize security, performance, and maintainability.
- If you are unsure, clearly state your uncertainty.
- Always respond in clear markdown format.

User Prompt (The Dynamic Request)

- This is what the **user** types.
- It changes with every message.
- It tells the AI **what to do right now**.
- **Example of User Prompts:**
- “Review this code: [code]”
- “Summarize the meeting notes”
- “Help me write a marketing email”

Common Prompt Frameworks

- **PTCF**
  - Persona, Task, Context, Format
  - Structured prompt building with role + output control
  - Agent design, consistent outputs, enterprise tasks
  - When to choose it: When you want consistent, professional, structured results (code reviews, reports, support tickets, analysis)
- **RISEN**
  - Role, Instructions, Steps, Expectations/End goal, Narrowing
  - Detailed process + constraints
  - Complex multi-step tasks, workflows, agents
  - PTCF vs RISEN: PTCF is cleaner and faster for most agent work. RISEN is better when the task is procedural/complex
- **COT**
  - Chain-of-Thought
  - Forcing step-by-step reasoning
  - Math, logic, complex problem-solving, analysis
  - Best combined with: PTCF or RISEN when the task requires deep analysis

PTCF

Component	What it is	Why it matters	Example
Persona	The identity/role/expertise of the AI	Sets tone, knowledge level, personality, and decision-making style	“You are a senior enterprise IT architect with 15+ years experience...”
Task	The core mission or what to do	Clearly defines the goal	“Analyze this system architecture and identify security risks...”
Context	Background information, constraints, environment	Gives the AI necessary situational awareness (audience, data, rules, limitations)	“This is for a Fortune 500 bank. Regulations include GDPR and SOC2...”
Format	Desired output structure	Ensures usable, consistent, parseable results	“Respond in a markdown table with columns: Risk, Severity, Mitigation...”

PTCF (Example)

Customer Support / Ticket Routing Agent

- You are an expert Tier-3 Customer Success Engineer at a SaaS company. You are empathetic, highly technical, and extremely precise in classifying issues. (Persona)
- Classify the following customer support ticket and recommend the best routing path and initial response. (Task)
- Ticket details: [paste full ticket text]. This customer is on the Enterprise plan, has been with us for 14 months, and their account manager is Sarah Chen. Our product is a project management tool. Common categories: Bug, Feature Request, Billing, Integration Issue, Onboarding. (Context)
- Respond using this exact format:
  - Classification: [Category]
  - Confidence: [High/Medium/Low]
  - Recommended Routing: [Team/Engineer name or queue]
  - Urgency: [Critical/High/Medium/Low]
  - Suggested First Reply (professional and empathetic, max 150 words)
  - Additional Information Needed: [list if any]

Code Review & Software Engineering Agent

- You are a Staff Software Engineer and code reviewer at Google with deep expertise in Python, TypeScript, and system design. You are strict about clean code, security, and performance. (Persona)
- Perform a thorough code review on the following pull request. (Task)
- The codebase is a FastAPI backend service handling user authentication and payments. Focus especially on security vulnerabilities, error handling, and scalability. Here is the code diff: [paste code]. This change is part of the Q3 security hardening initiative. (Context)
- Structure your review exactly like this:
  - Overall Assessment (one sentence)
  - Critical Issues (if any)
  - Major Issues
  - Minor Issues / Suggestions
  - Positive Observations
  - Recommended Refactor (if needed)
  - Final Verdict: Approve / Approve with changes / Request changes

PTCF

- Persona is usually placed in the System Prompt (the agent’s “constitution”)
- Task + Context + Format often go in the User Prompt
- Be as specific as possible in Context — this is where most people under-deliver

Template of PTCF

You are [Detailed Persona/Role + Expertise + Personality Traits].
You have [X] years of experience in [field].
Your style is [describe tone, approach, strengths].
You always [key principles or values, e.g., prioritize accuracy, think step-by-step, etc.]. (Persona)

[Main Goal / What you want the agent to do] (Task)

[All relevant background information, constraints, audience, tools available, policies, previous context, data, etc.] (Context)

Respond using the following exact format:
[Specify the desired output structure in detail]
- Section 1: ...
- Section 2: ...
- Use markdown tables where appropriate
- Use bullet points / numbered lists
- Include confidence levels if relevant
- etc.

[Additional rules, e.g., "Do not hallucinate facts", "Always cite reasoning", "Keep total response under 800 words", etc.]

P

T

C

F

Template: PTCF with COT

```
You are [Detailed Persona/Role + Background + Expertise + Personality Traits].
You have [X] years of experience in [field/industry].
You are known for being [key traits: extremely analytical, meticulous, creative, objective,
You always think step-by-step and prioritize accuracy over speed. (Persona)

[Clear, single-sentence main objective of the task] (Task)

[Provide all necessary background information, constraints, data, audience, policies, tools

Think step by step before giving your final answer.
Use the following reasoning process:

1. Understand the problem and goal
2. Analyze the given information and identify key factors
3. Consider possible approaches or solutions
4. Evaluate pros, cons, risks, and implications
5. Synthesize the best recommendation or output

Then, respond using this exact format:

[Desired structured output]
- Section 1: ...
- Section 2: ...
- Use tables, bullet points, or numbered lists where helpful
- Include confidence level where appropriate

Additional rules:
- Be concise but thorough
- Base all conclusions on the provided context
- Explicitly state any assumptions you make
- Do not hallucinate information
```

COT

It dramatically improves reasoning quality, reduces hallucinations, and makes the thinking process transparent

Example of PTCF with COT

You are a Staff Software Engineer at Google with 12+ years of experience. You specialize in Python, system design, and security. You are meticulous, pragmatic, and excellent at balancing clean code with business needs. (Persona)

Perform a comprehensive code review on the following pull request. (Task)

The code is part of a FastAPI backend handling user authentication. Focus on security, performance, and maintainability. Here is the code diff: [paste code]. This is for a fintech product under PCI-DSS compliance. (Context)

Think step by step before giving your final answer. Use this reasoning process:

1. Understand the problem and goal
2. Analyze the given information and identify key factors
3. Consider possible approaches or solutions
4. Evaluate pros, cons, risks, and implications
5. Synthesize the best recommendation or output

Then, respond using this exact format:

- Overall Assessment
- Critical Issues (if any)
- Major Issues
- Minor Issues & Suggestions
- Positive Observations
- Recommended Changes (with code snippets if needed)
- Final Verdict: [Approve / Approve with Changes / Request Changes]
- Confidence: [High/Medium/Low]

RISEN

What is RISEN

Act as a [Role + Expertise + Personality]. (Role)

[Clear instructions on what you want the AI to do]. (Instructions)

Follow these exact steps to complete the task:
1. [Step 1]
2. [Step 2]
3. [Step 3]
... (Steps)

The end goal is to produce [specific deliverable]. (End Goal / Expectations)

Additional constraints:
- [Narrowing rules: tone, length, things to avoid, focus areas, etc.]

Example

Act as a Staff Software Engineer at Google with 12+ years of experience specializing in backend systems, security, and clean code. You are meticulous, pragmatic, and constructive in your feedback. (Role)

Perform a thorough code review on the provided pull request. (Instructions)

Follow these exact steps:
1. First, understand the purpose of the changes and the context of the codebase.
2. Analyze the code for security vulnerabilities, performance issues, maintainability, and adherence to best practices.
3. Identify critical, major, and minor issues.
4. Suggest specific improvements or refactors where needed.
5. Summarize positive aspects of the code. (Steps)

The end goal is to deliver a professional, actionable code review that the developer can immediately use to improve the code. (End Goal)

Additional constraints:
- Do not approve any code with critical security issues.
- Keep the total response under 600 words.
- Be direct but professional — no sarcasm.
- Base all feedback only on the provided code and context.
- Use markdown formatting for readability. (Narrowing)

Template of RISEN

Act as a [Specific Role + Expertise + Personality Traits]. (Role)

[Clear, direct instruction about what you want the AI to accomplish]. (Instructions)

Follow these exact steps in order:
1. [First step]
2. [Second step]
3. [Third step]
4. [Fourth step]
... (add as many steps as needed) (Steps)

The end goal / expected output is [describe the final deliverable clearly]. (End Goal / Expectations)

Additional constraints and rules:
- [List any limitations, tone requirements, things to avoid, length, focus areas, etc.] (Narrowing)

Example of RISEN with COT

Act as a [Specific Role + Expertise + Personality Traits]. (Role)

[Clear, high-level instruction about the main objective]. (Instructions)

Think step by step and use Chain-of-Thought reasoning throughout the process. For every major decision or analysis, explicitly show your reasoning.

Follow these exact steps in order:

1. [Step 1 - with reasoning]
2. [Step 2 - with reasoning]
3. [Step 3 - with reasoning]
... (Steps)

The end goal is to produce [specific deliverable or outcome]. (End Goal / Expectations)

Additional constraints and rules:
- [Tone, length, things to avoid, quality standards, etc.] (Narrowing)

Typical types of prompts

- Zero shot
  - These prompts are used to perform a task without any prior training data.
- Few shot
  - These prompts are used to perform a task with a small amount of training data.

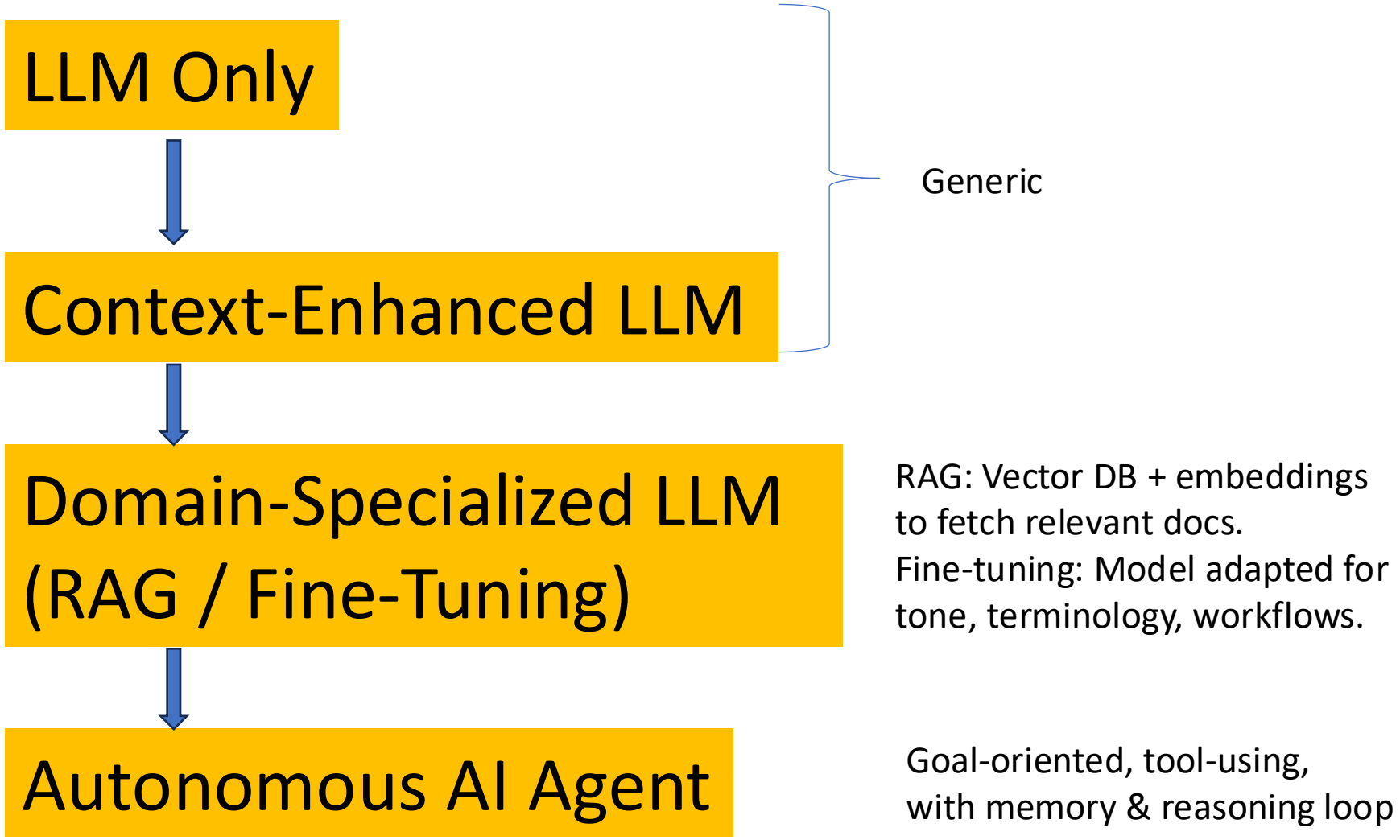
Few shot-Example

```
以下是一個預算計算的例子
----------------------
例子：
場地租賃費 (Venue Rental Fee)=0
器材購置費 (Equipment Purchase Cost)=3000
教練費 (Coaching Fee)=350
教練時間 (Coaching Time)=20
宣傳費用 (Promotion Expenses)=2000
保險費用 (Insurance Expenses) =1500
總支出：13500
--------------------
現在支出更新如下：
場地租賃費 (Venue Rental Fee)=9000
器材購置費 (Equipment Purchase Cost)=3000
教練費 (Coaching Fee)=390
教練時間 (Coaching Time)=20
宣傳費用 (Promotion Expenses)=2000
保險費用 (Insurance Expenses) =1500
-------------
請用第一個例子去計算這次的預算是多少.
```

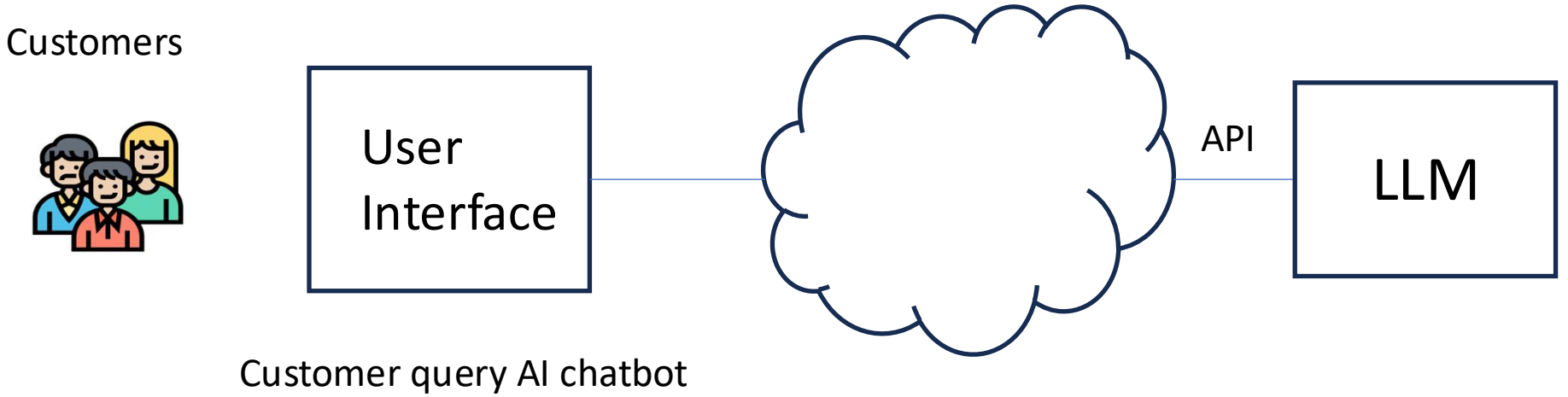
Example

Agentic AI Solutions

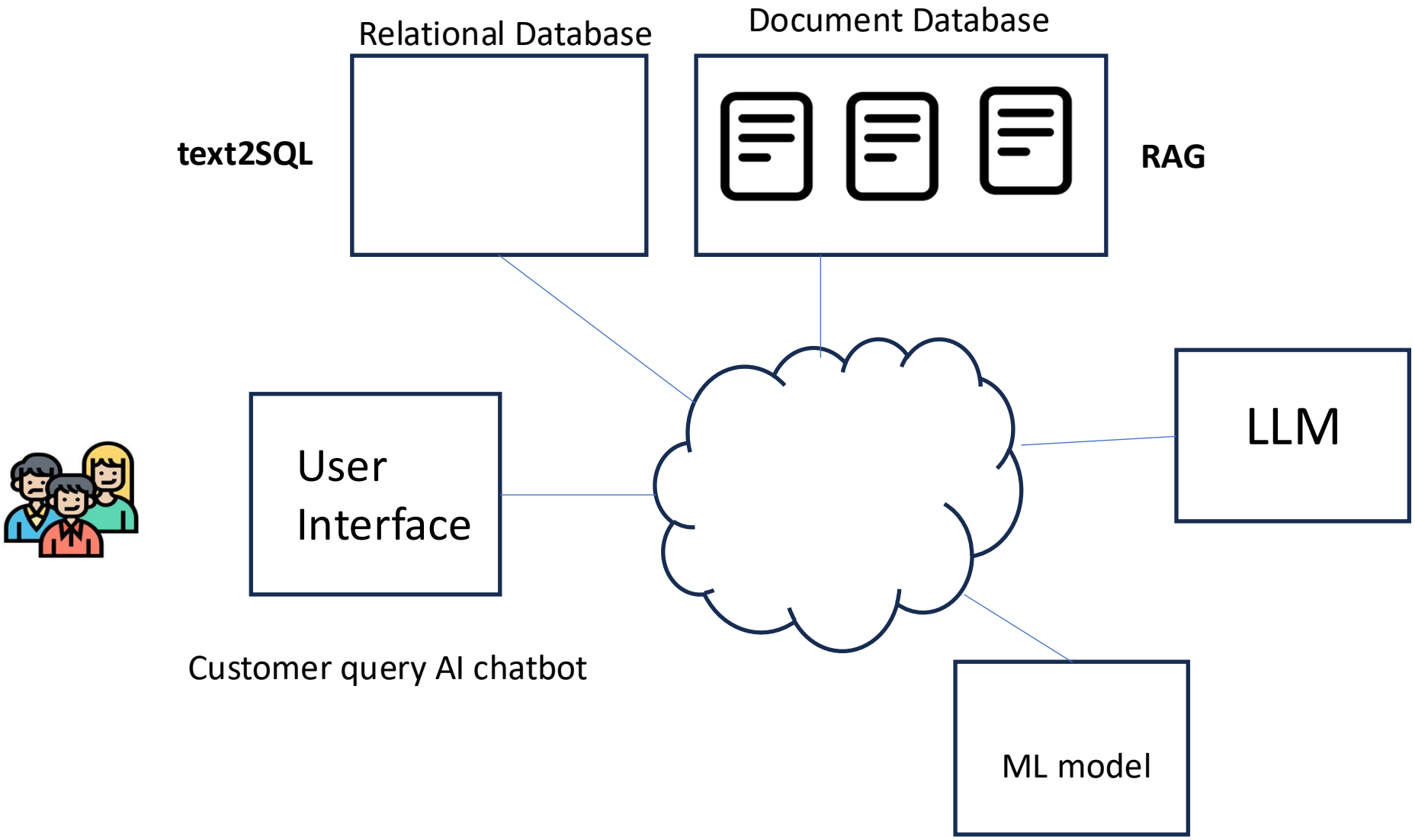
Evolution of AI Applications with LLM



AI Solutions with LLM
Limitation of using generic LLM for AI Chatbot

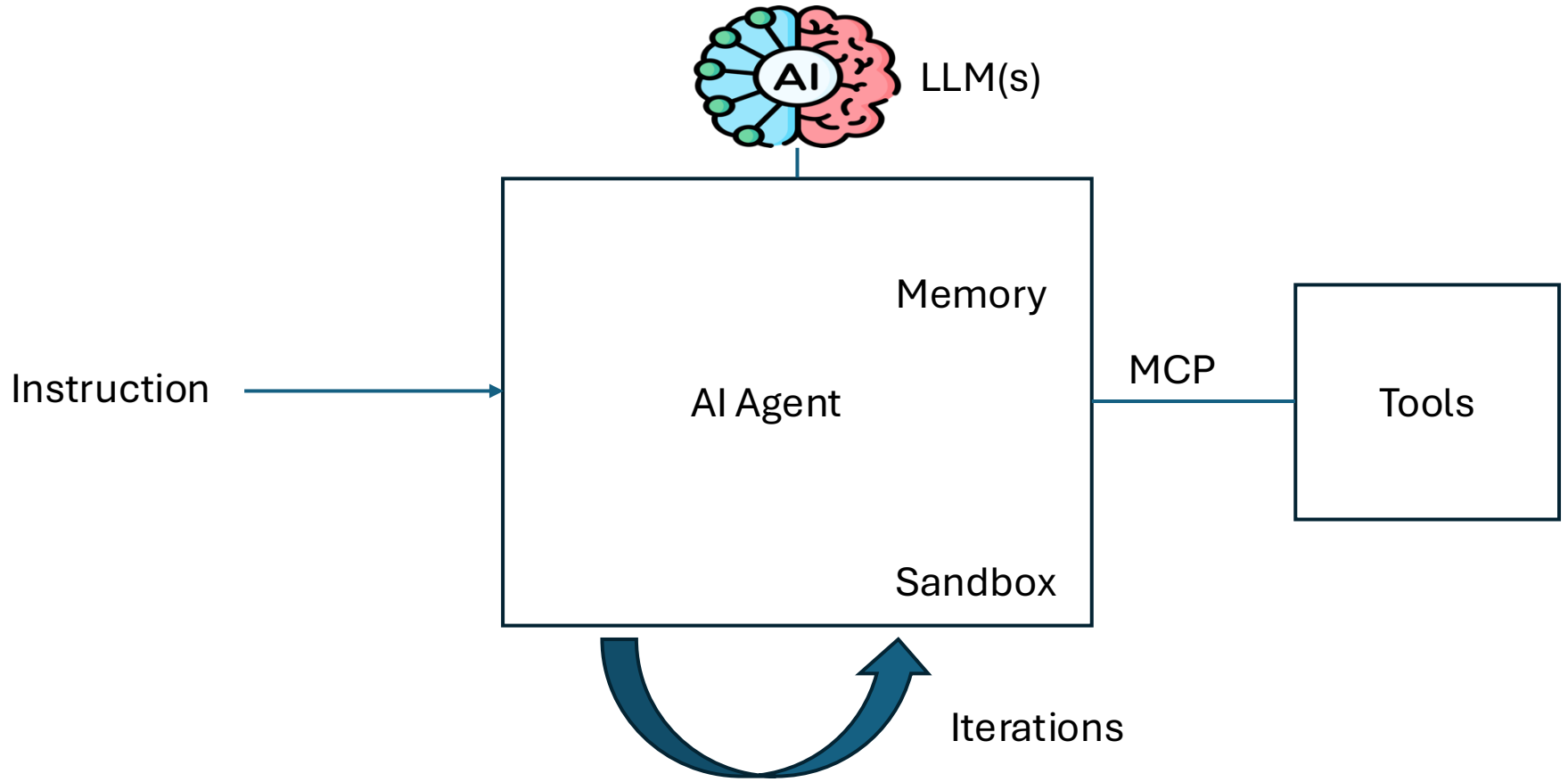


AI Solutions with LLM
AI Chatbot (Sample)



Customer query AI chatbot

Agentic Development Framework



AI Agent=LLM Model + Harness